GATE
CS & DA

Artificial Intelligence

Informed Search

Lecture No. 03

By- SIDDHARTH SABHARWAL SIR

Recap of Previous Lecture

- Topic
- Topic
- Topic
- Topic
- Topic

A*

Questions

Topics to be Covered

Topic — weighted A*

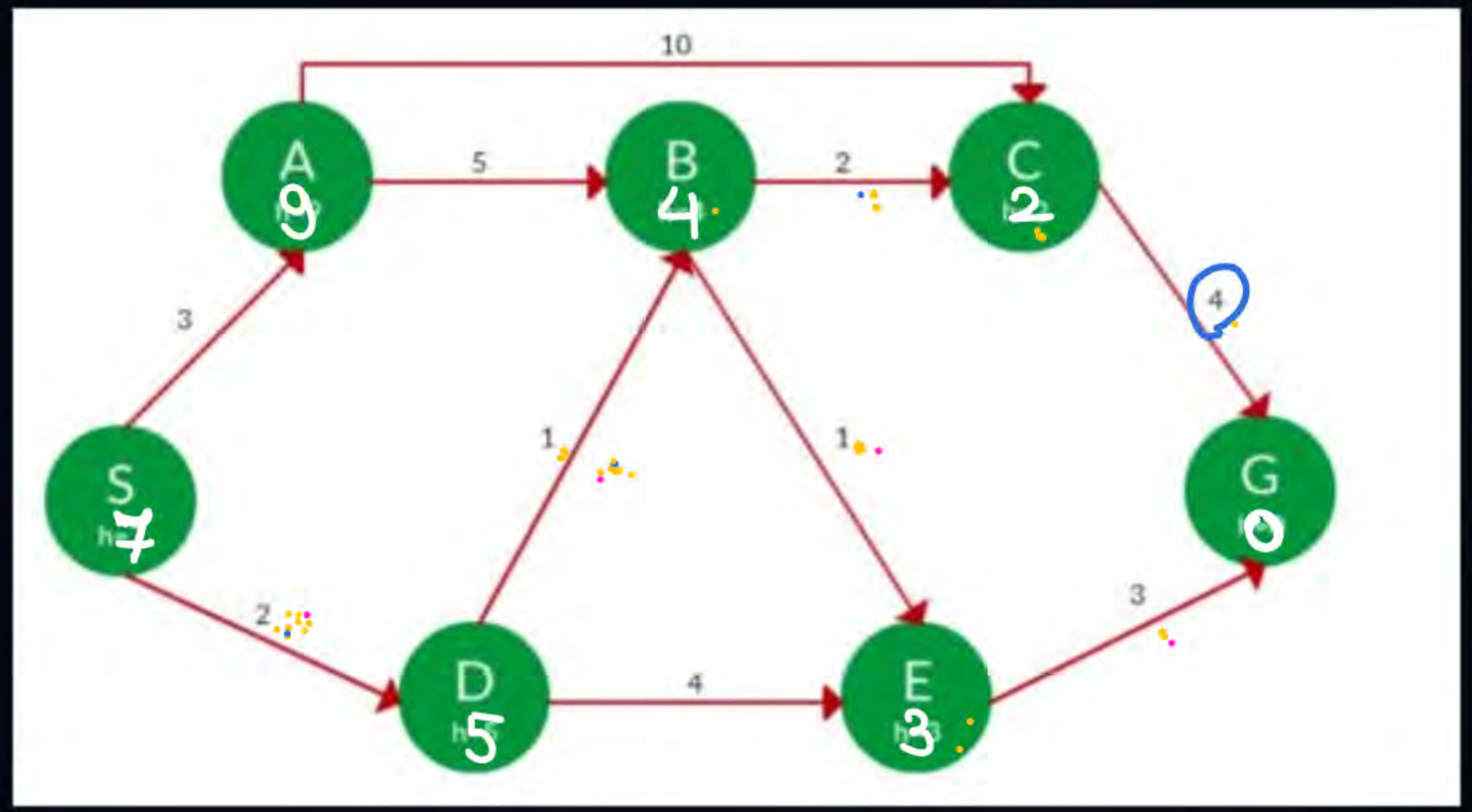
Topic — AO*

Topic — AO* Vs A*

Topic

Topic

A* algorithms



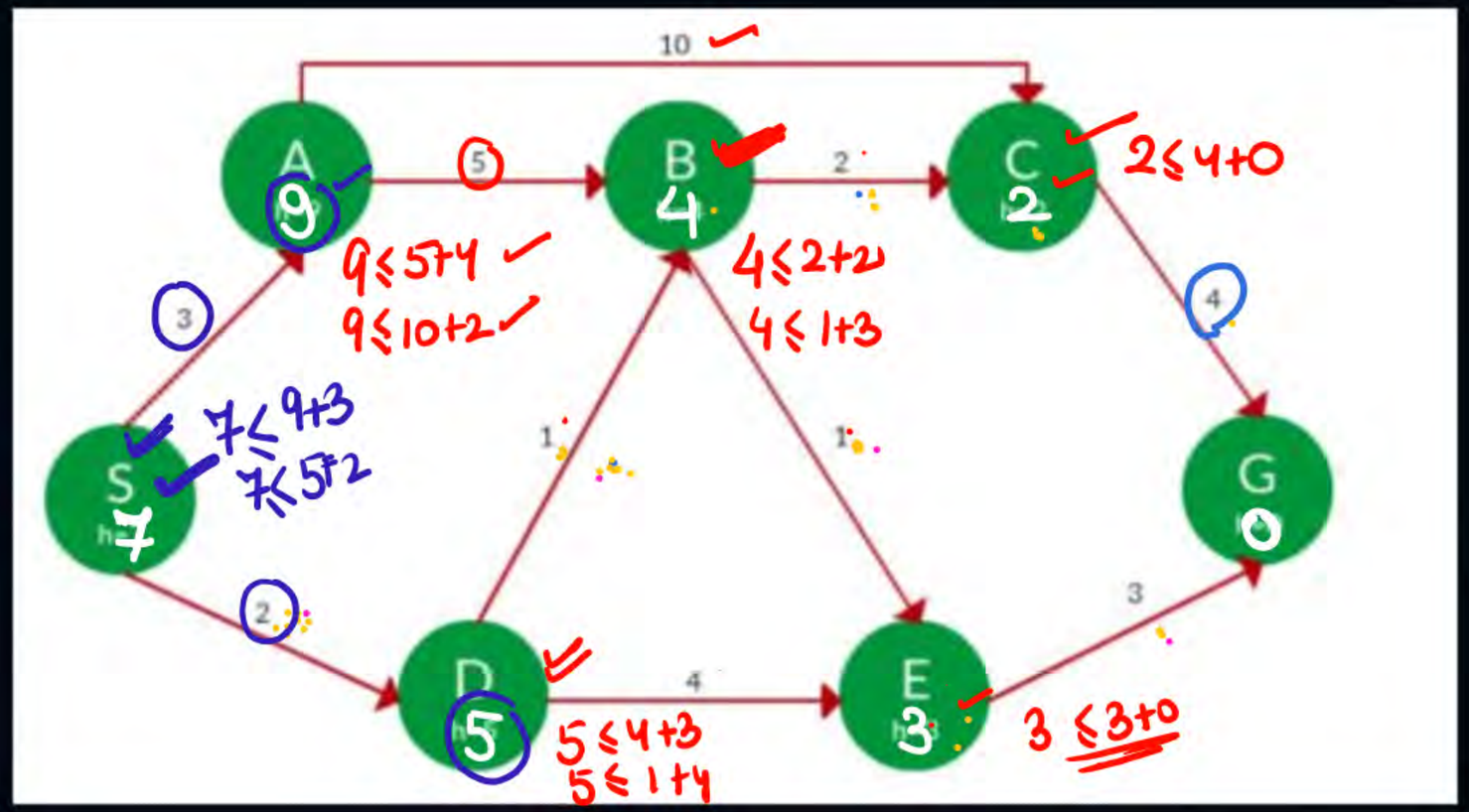
- S-G Cost
- admissible heuristic?
- Consistent ?

	S	D	B	C	E
S	0	0	0	0	0
A	12	12	12	12	12
B	∞	7	7	7	7
C	∞	∞	7	7	7
D	7	7	7	7	7
E	∞	9	7	7	7
G	∞	∞	∞	7	7
				9	7

E ← G

B → D → S

A* algorithms



- S-G Cost
- admissible heuristic?

Consistent

$A^* \Rightarrow f(n) = g(n) + h(n)$

Weighted A* (wA*) Algorithm

- Introduction of a Weight Factor: Weighted A* modifies the A* algorithm by introducing a weight www to the heuristic function:

$f(n)=g(n)+w \cdot h(n)$ we multiply weight with heuristic fxn.

A^* = mix of UCS + GBFS

- Here, w is a constant (typically $w \geq 1$).
- Effect of the Weight ~~www~~:

If w=1: A* behaves exactly like the standard A* algorithm, ensuring optimality.

If w>1: The heuristic becomes more influential, making the algorithm more aggressive in reaching the goal. This typically results in faster search times but may lead to suboptimal solutions.

making it a little faster.

Weighted A* (wA*) Algorithm

Edge Costs:

- S to A: 2
- S to B: 3
- S to C: 4
- A to D: 4
- B to D: 1
- B to E: 3
- C to E: 2
- D to G: 5
- E to G: 2
- F to G: 1
- A to F: 6
- C to F: 2

Heuristic Values (h):

- $h(S) = 6$
- $h(A) = 4$
- $h(B) = 3$
- $h(C) = 2$
- $h(D) = 4$
- $h(E) = 2$
- $h(F) = 2$
- $h(G) = 0$

$w = 2$ $f = g + 2h$

Find Path

	S	C	B	A	E	G
S	0	0	0	0	0	
A	10	10	10	10	10	
B	9	9	9	9	9	
C	8	8	8	8	8	
D	∞	∞	12	12	12	
E	∞	10	10	10	10	
F	∞	10	10	10	10	
G	∞	∞	∞	∞	8	

S→C→E→G

(WA* non optimal)

Stop

Weighted A* (wA*) Algorithm

GBFS → Path & Cost ⇒ S → B → F → G ⇒ (8 Cost)

A* → Path & Cost

Find Path → S → C → A → E → G (Cost 7)

	S	B	A	C	D	E	F	G
S	0	0	0	0	0	0	0	0
A	6	6	6	6	6	6	6	6
B	4	4	4	4	4	4	4	4
C	6	6	6	6	6	6	6	6
D	∞	8	8	8	8	8	8	8
E	∞	8	8	8	8	8	8	8
F	∞	∞	10	8	8	8	8	8
G	∞	∞	∞	8	9	8	7	7

	S	C	A	E	F	G
S	0	0	0	0	0	0
A	10	10	10	10	10	10
B	9	9	9	9	9	9
C	8	8	8	8	8	8
D	∞	∞	12	12	12	12
E	∞	10	10	10	10	10
F	∞	10	10	10	10	10
G	∞	∞	∞	8	10	10

Weighted A* (wA*) Algorithm

A*

	S	B	D	A	C	G
S	0	0	0	0	0	
A	6	6	6	6	6	
B	4	4	4	4	4	
C	6	6	6	6	6	
D	∞	5	5	5	5	
E	∞	13	13	13	13	
F	∞	∞	∞	14	12	
G	∞	∞	7	7	7	

S → B → D → G

non optimal
Bcoz heuristic poor.

Stop

* A* algo will give optimal result when heuristic are Consistent

* In A*, WA* we expand nodes as per the fxn values and when we reach goal node we stop.

⇒ I J K M ⇒ goal

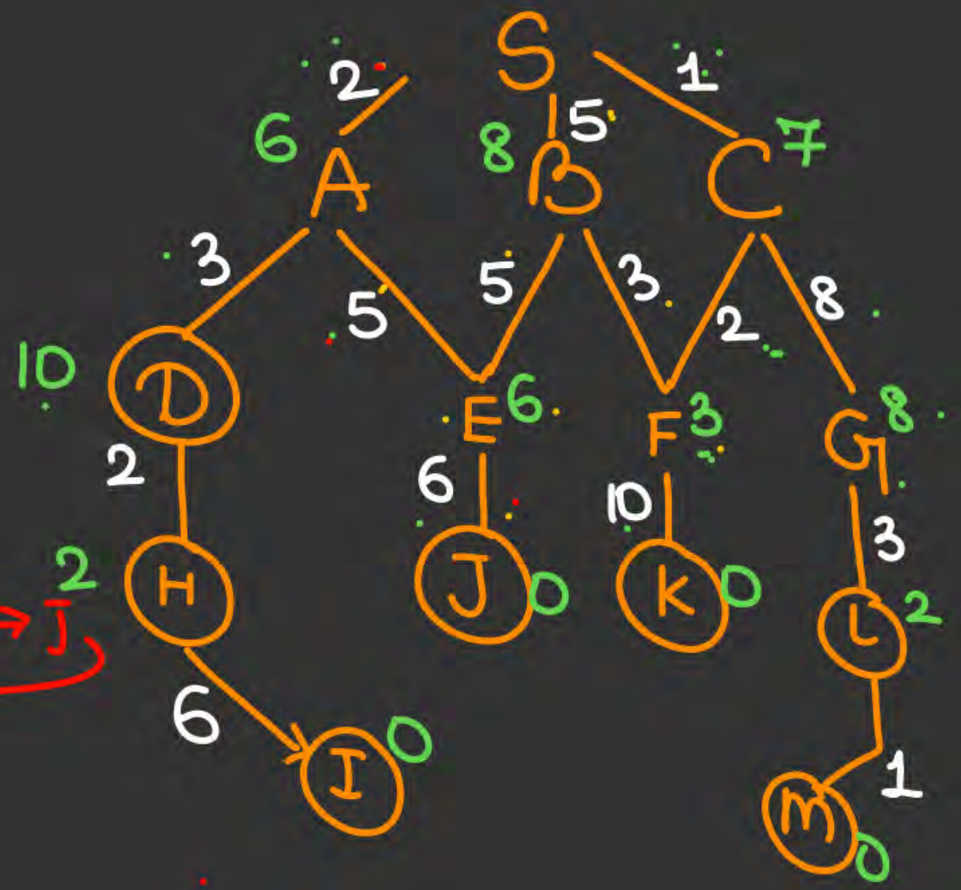
- GBFS ⇒ S → A → E → J ✓

	S	A	C	F	B	E	J
S	0	0	0	0	0	0	
A	8	8	8	8	8	8	
B	13	13	13	13	13	13	
C	8	8	8	8	8	8	
D	∞	15	15	15	15	15	
E	∞	13	13	13	13	13	
F	∞	∞	6	6	6	6	
G	∞	∞	17	17	17	17	
H	∞	∞	∞	∞	∞	∞	
I	∞	∞	∞	∞	∞	∞	
J	∞	∞	∞	∞	∞	∞	
K	∞	∞	∞	∞	∞	13	
L	∞	∞	∞	13	13	13	
M	∞	∞	∞	∞	∞	∞	

- A*

S → A → E → J

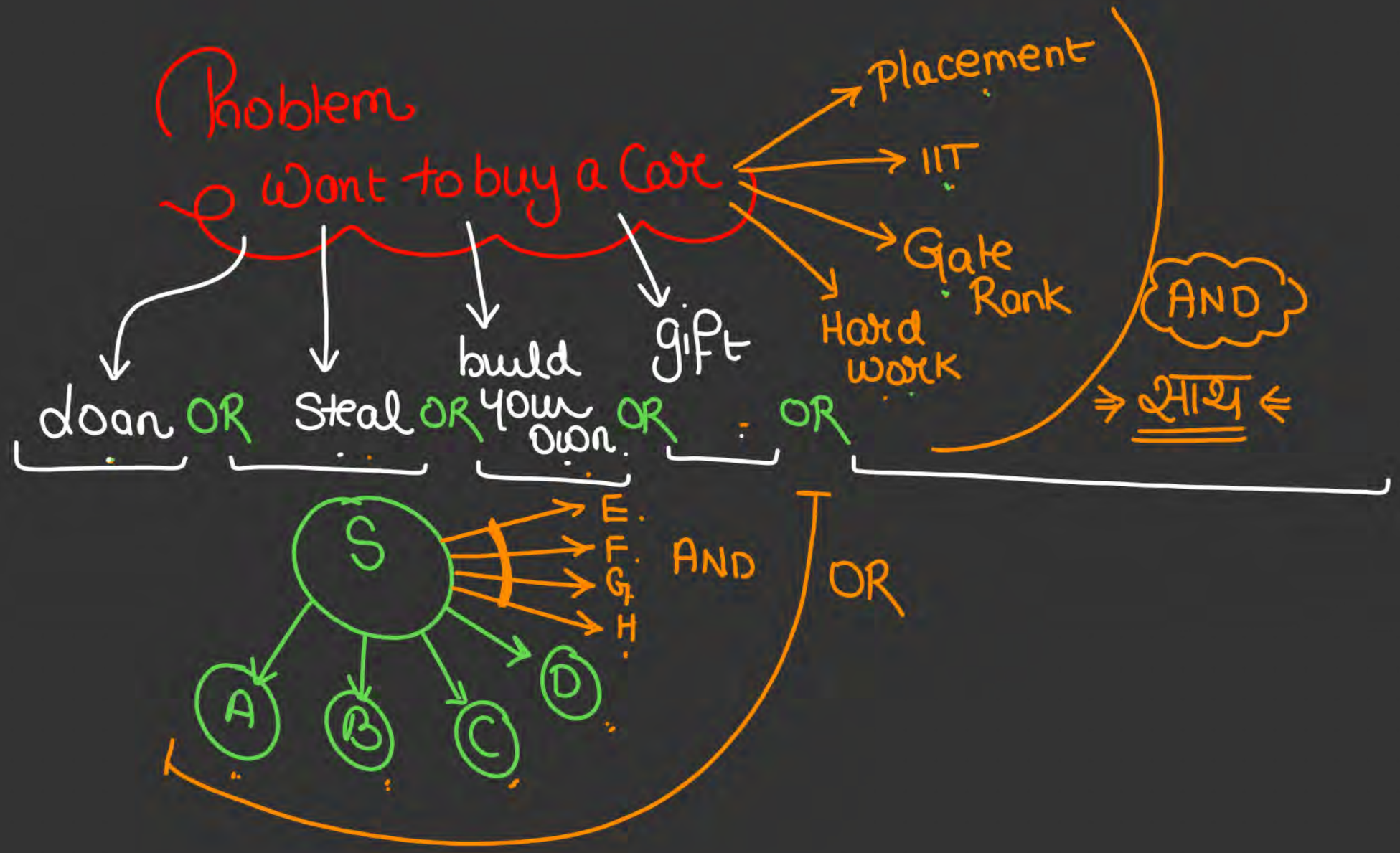
13



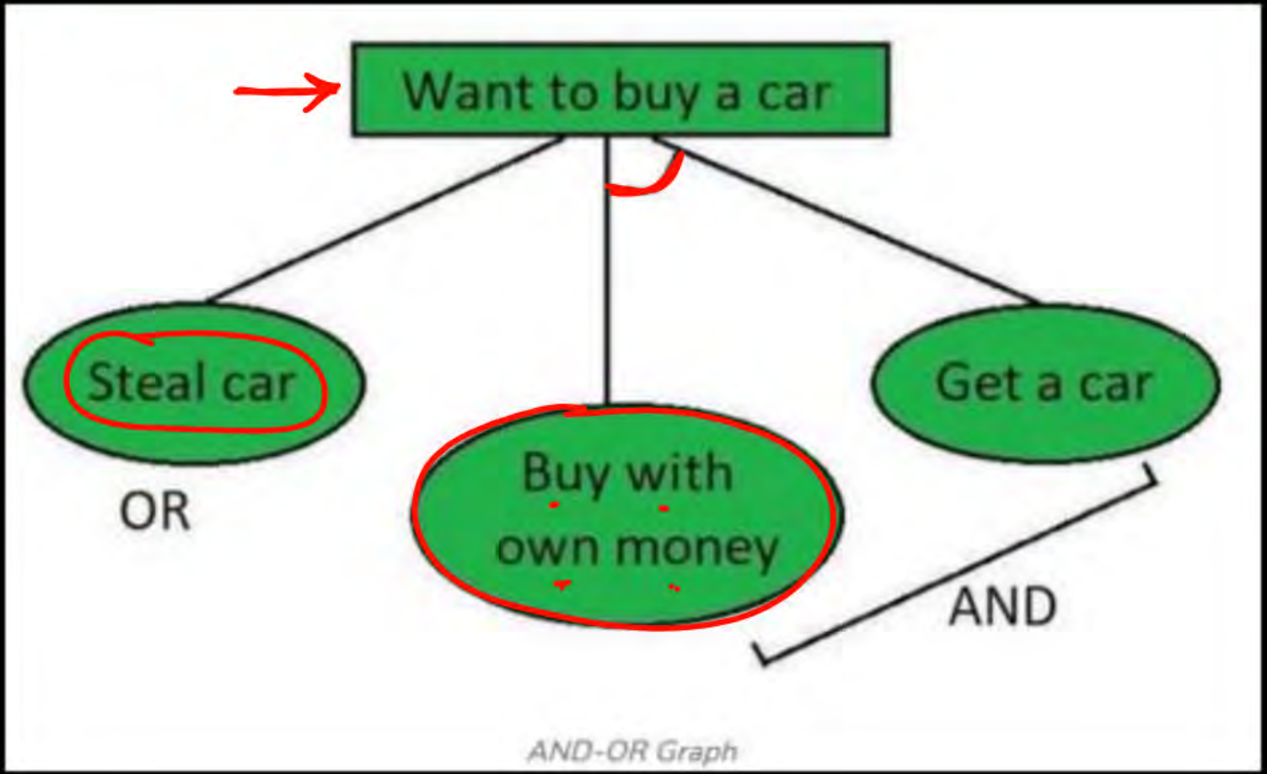
AO* algorithm – Artificial intelligence

(And OR) * Algo.

- The AO* method divides any given difficult problem into a smaller group of problems that are then resolved using the AND-OR graph concept.
- AND OR graphs are specialized graphs that are used in problems that can be divided into smaller problems.
- The AND side of the graph represents a set of tasks that must be completed to achieve the main goal, while the OR side of the graph represents different methods for accomplishing the same main goal.


Problem
Want to buy a Car
loan
OR
Steal
OR
build your own
OR
gift
OR
Hard work
Gate Rank
IIT
Placement
AND
⇒ साथ ⇐
S
A
B
C
D
E
F
G
H
AND
OR

AO* algorithm – Artificial intelligence

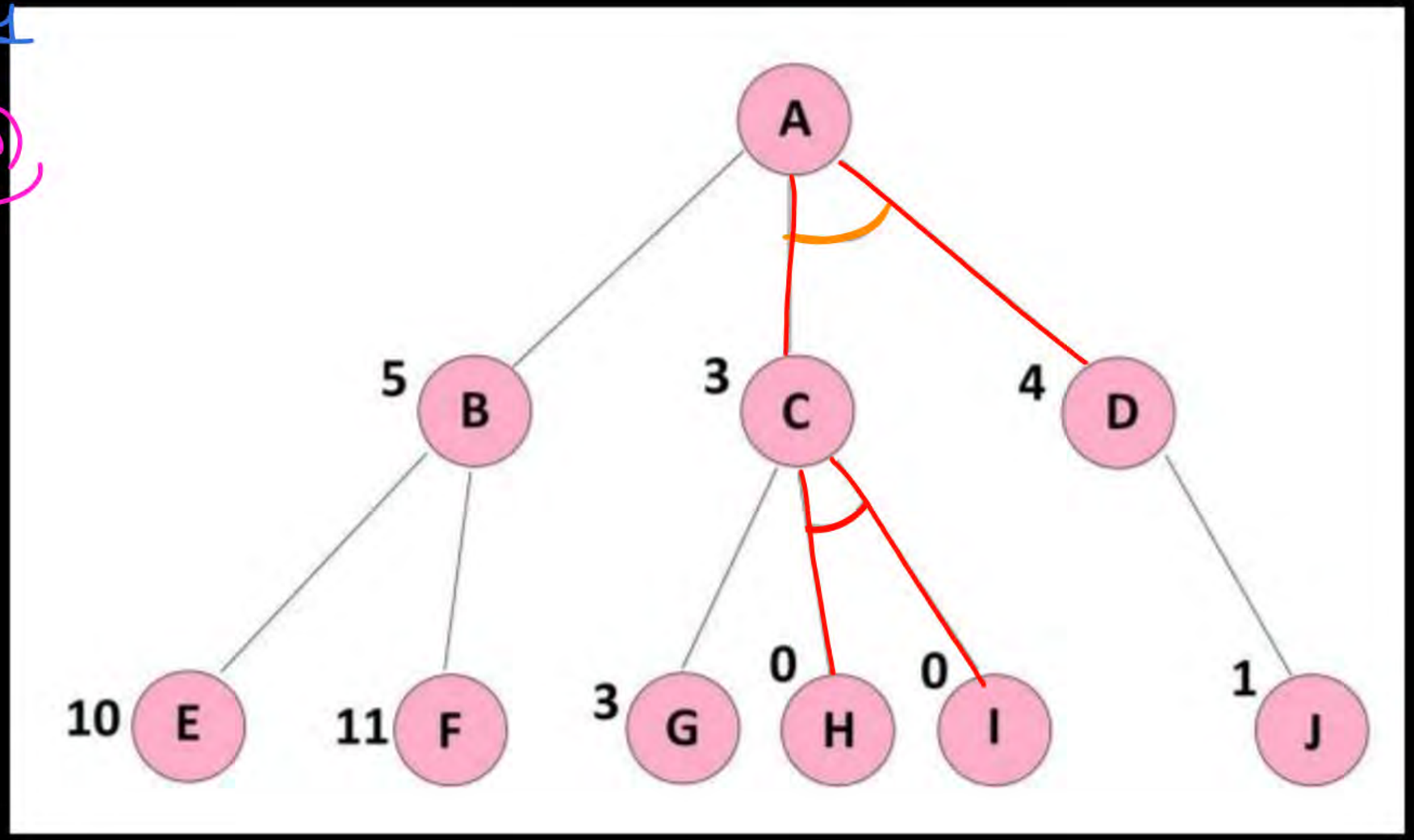


AO* algorithm – Artificial intelligence

- Here we have the concept of forward and backward propagation

- Forward Propagation

- Value written @node ⇒ heuristic
- Cost of edge = 1
- $f(n) = g(n) + h(n)$



- Forward Propagation

edge Cost = 1

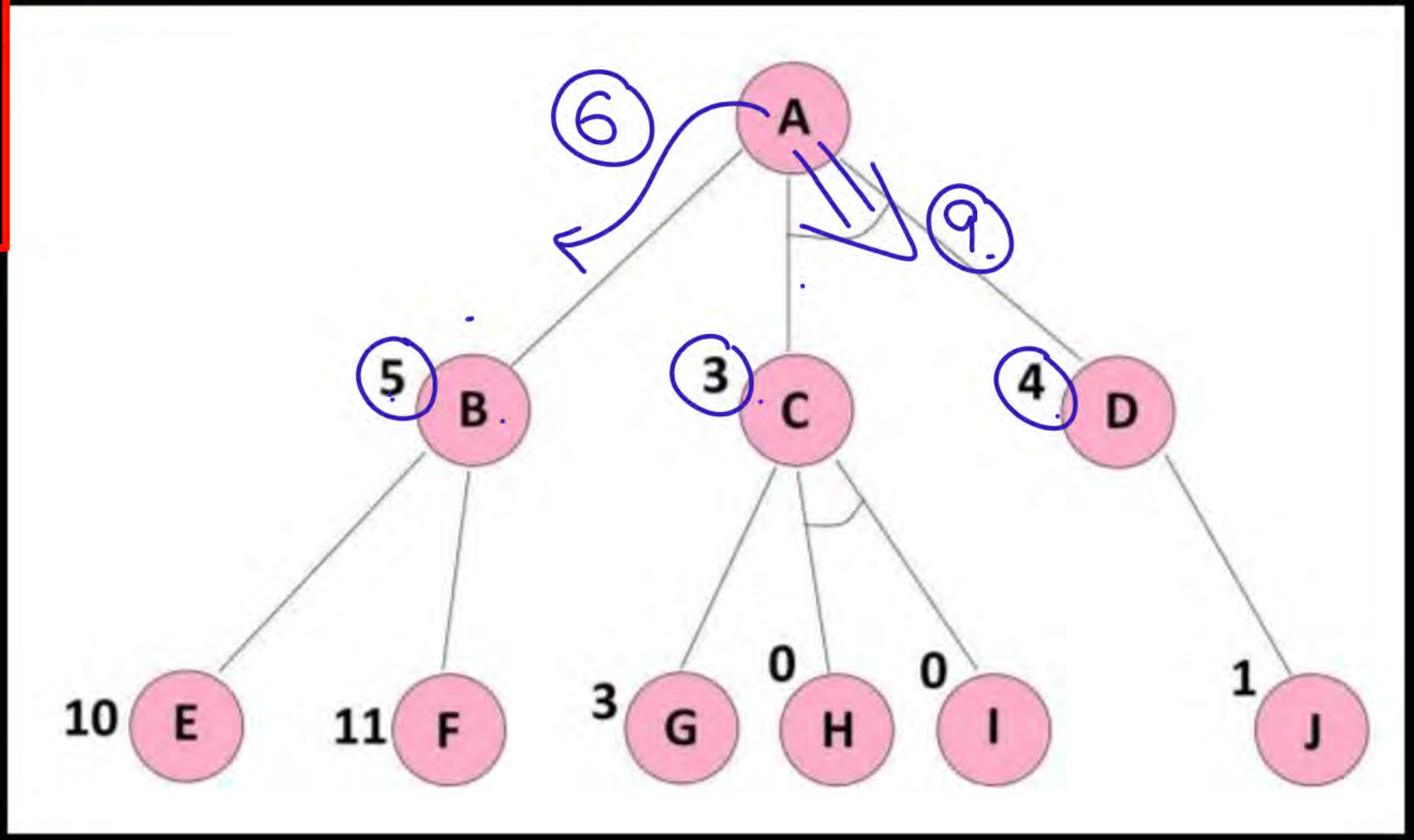
⇒ A → B ⇒ 1+5 ⇒ 6 → Choose(A→B)

⇒ A → C $D ⇒ 9

A → C ⇒ 1+3

A → D ⇒ 1+4

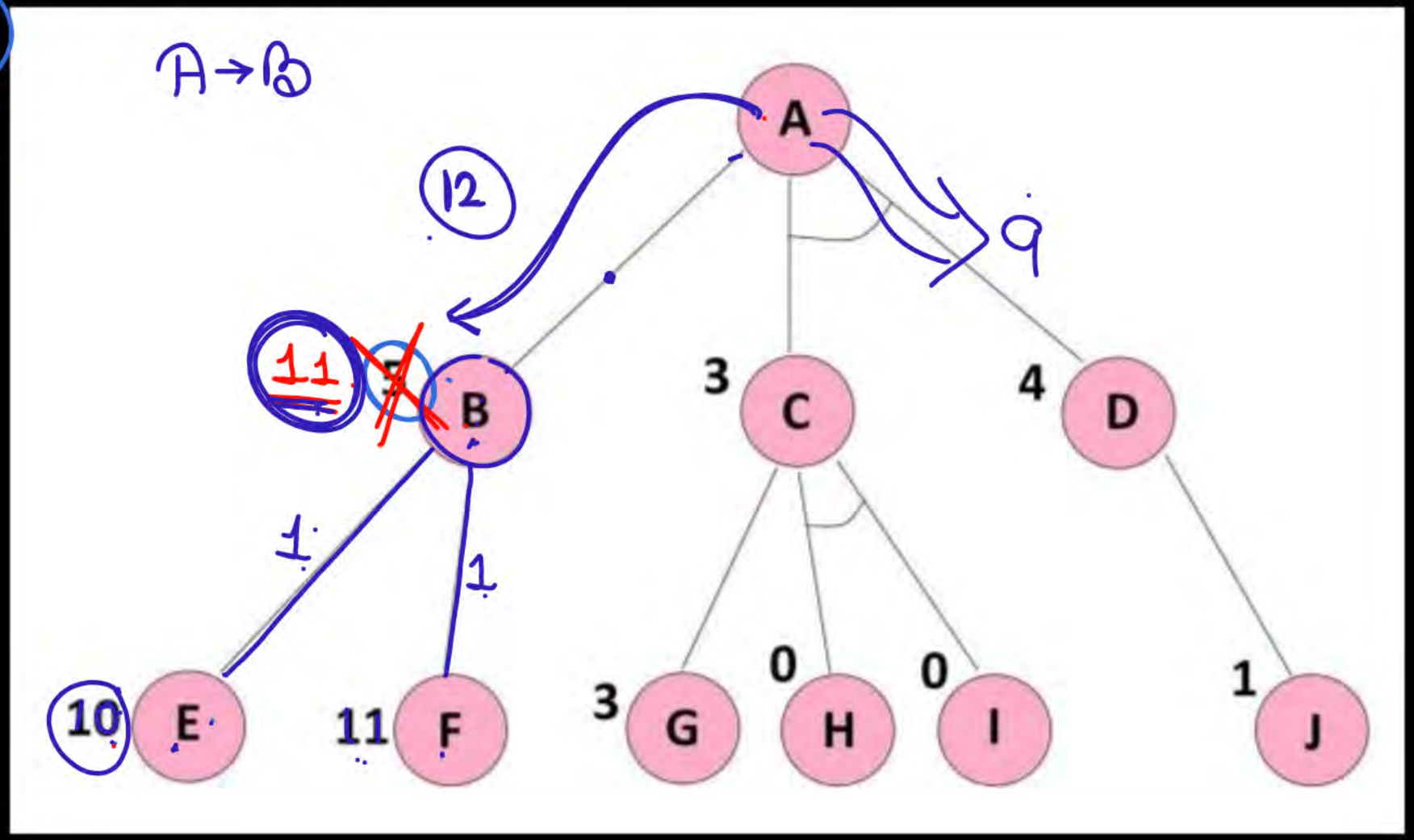
9



- Reaching the Last Level and Back Propagation

Now A → B

B → E ⇒ 1+10
= 11.
OR
B → F ⇒ 1+11
⇒ 12



- Reaching the Last Level and Back Propagation

$D \to J \Rightarrow 2$

$A \to (C\$D)$ 6

$C \to$ goal 2

$D \to$ goal 2

Now after going to C$D

we can move ahead through any C, D as heuristic=2

$(A \to C, D)$ 9

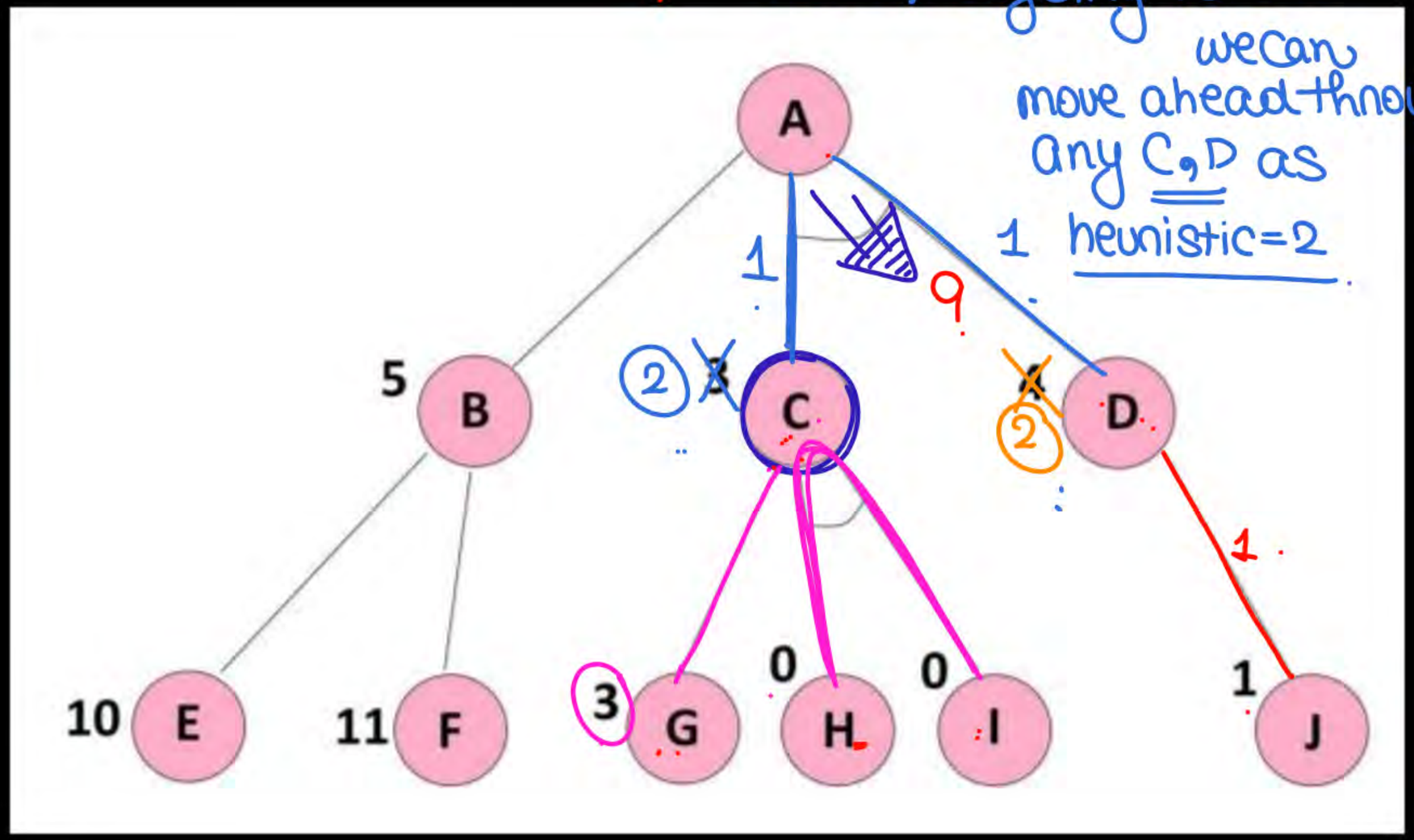
4 ← $C \to G$

OR

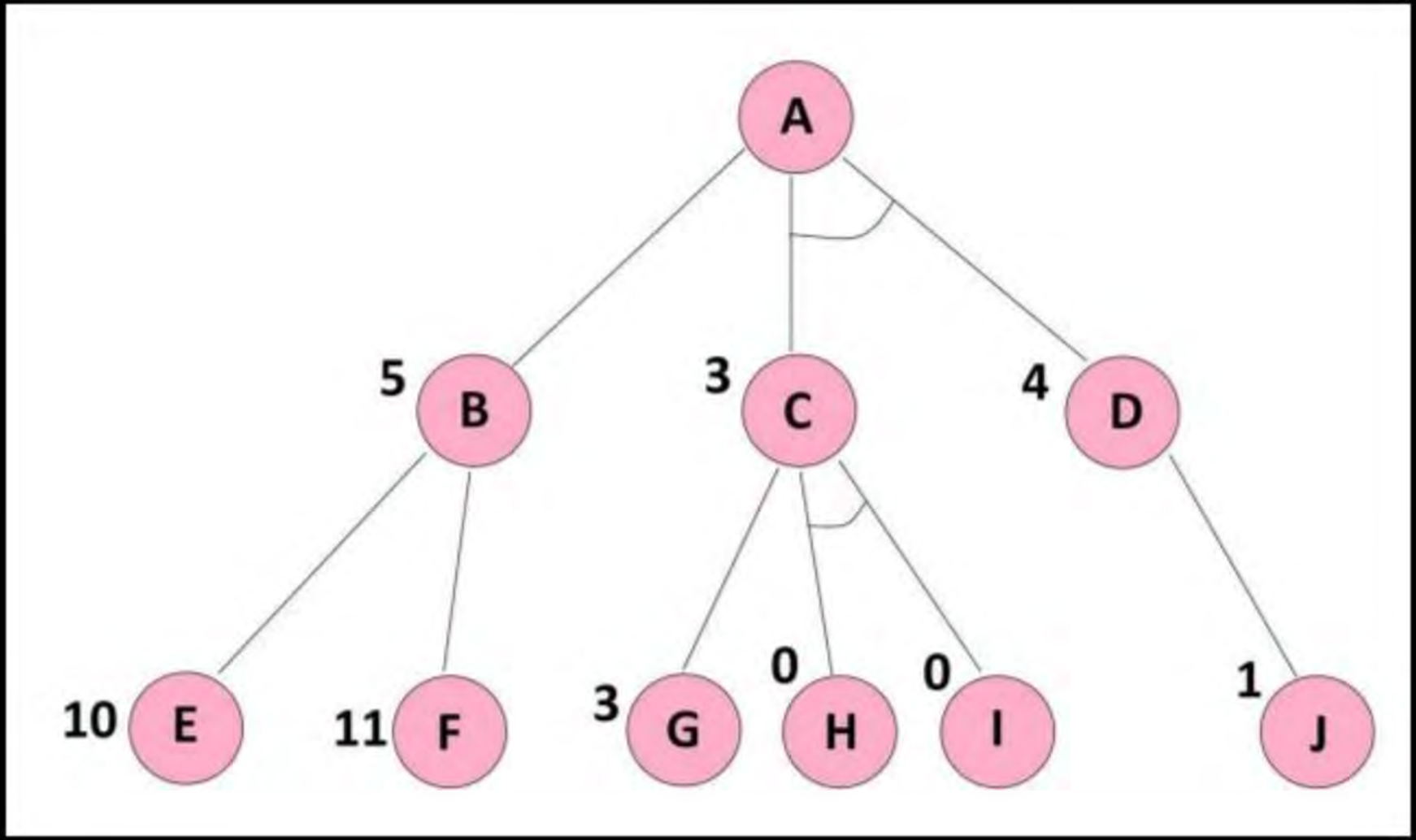
2 ← $C \to H\$I$

$C \to H + C \to I$

$1+1=2$

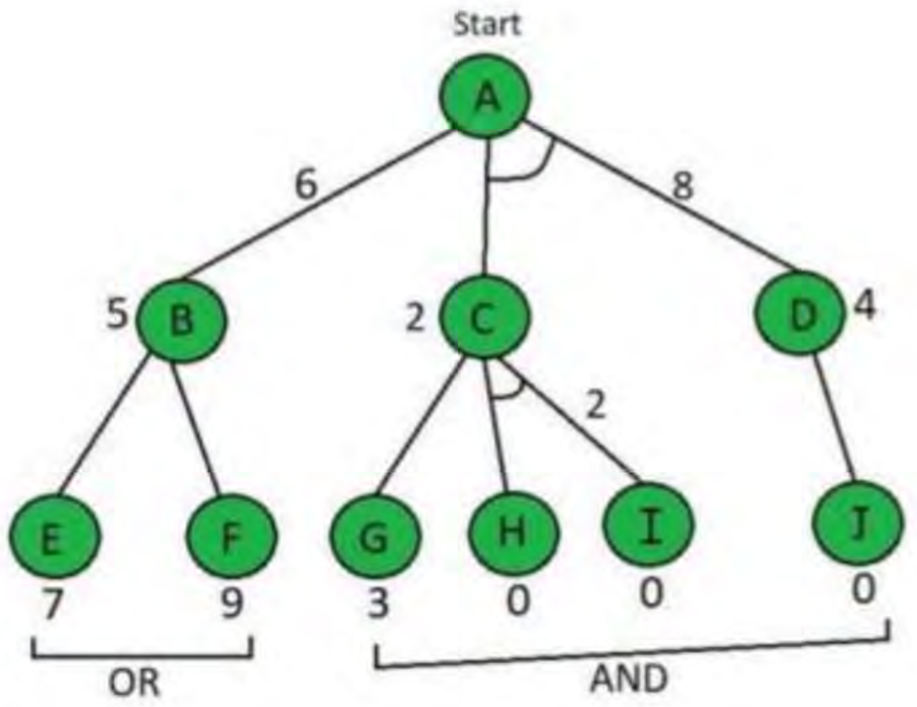


- Reaching the Last Level and Back Propagation

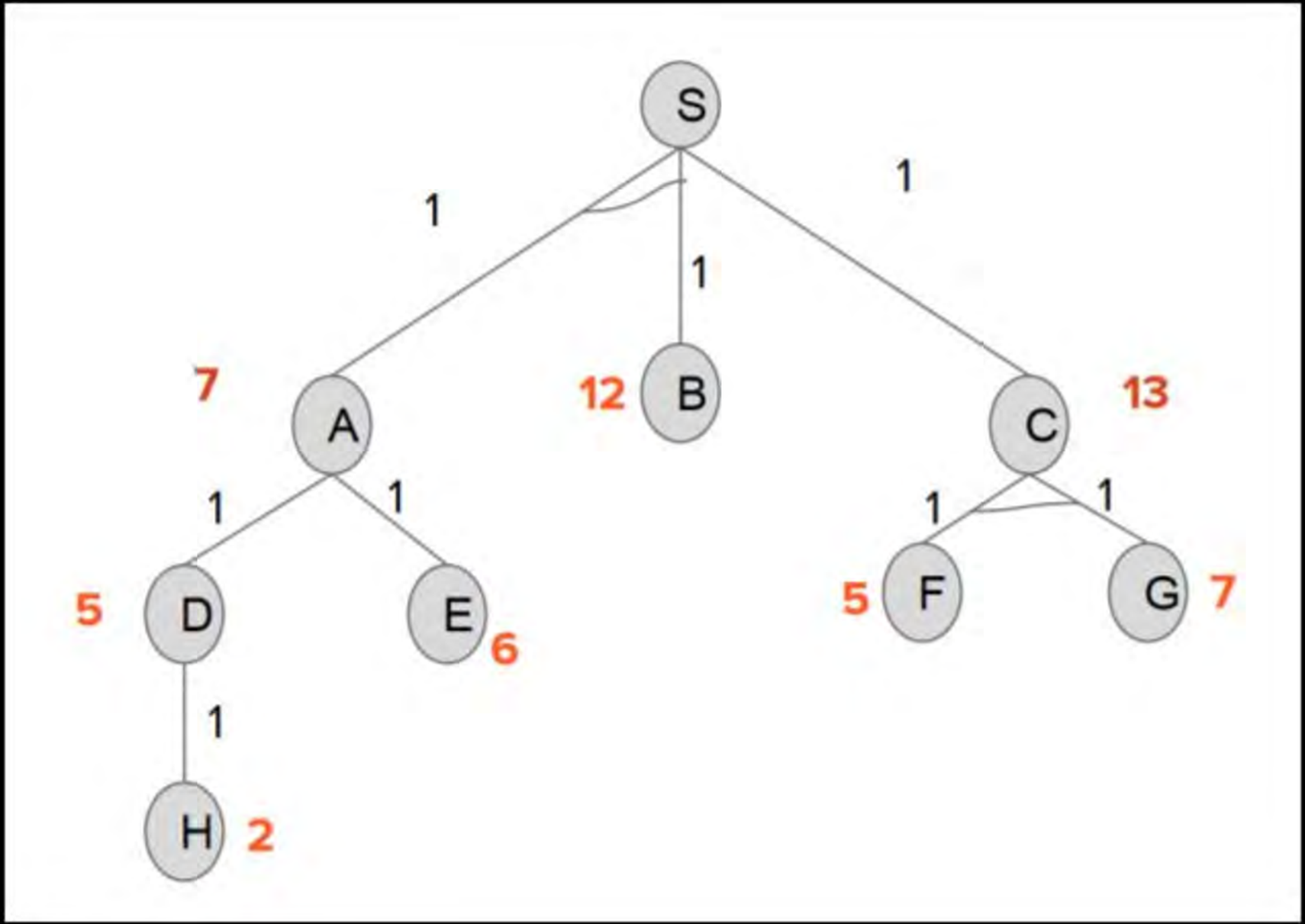


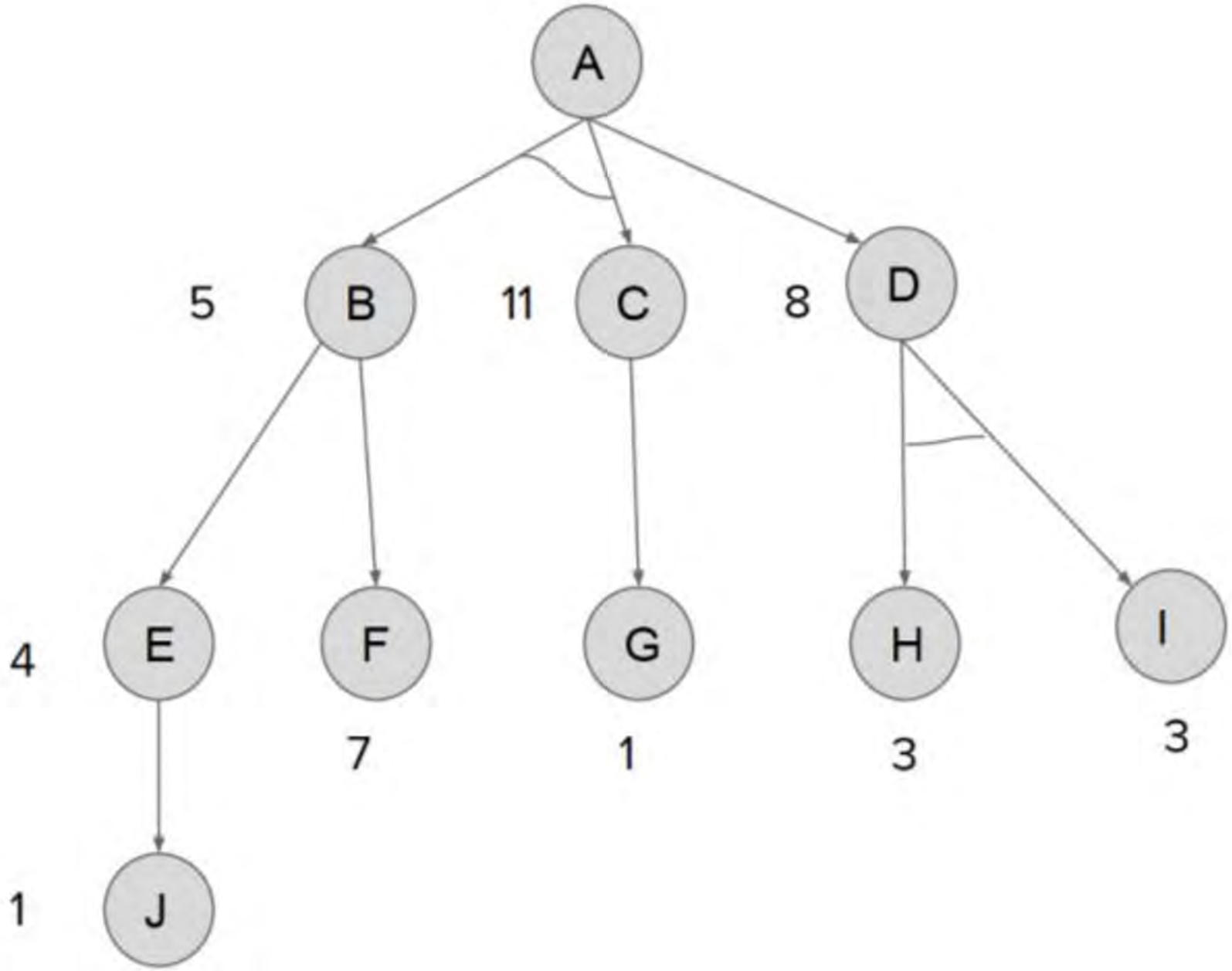
AO* algorithm – Artificial intelligence

- In the above figure, the buying of a car may be broken down into smaller problems or tasks that can be accomplished to achieve the main goal in the above figure, which is an example of a simple AND-OR graph. The other task is to either steal a car that will help us accomplish the main goal or use your own money to purchase a car that will accomplish the main goal.
- The AND symbol is used to indicate the AND part of the graphs, which refers to the need that all subproblems containing the AND to be resolved before the preceding node or issue may be finished.
- The start state and the target state are already known in the knowledge-based search strategy known as the AO* algorithm, and the best path is identified by heuristics. The informed search technique considerably reduces the algorithm's time complexity. The AO* algorithm is far more effective in searching AND-OR trees than the A* algorithm.



AO Algorithm – Question tree*


S
1
1
1
7
A
12
B
C
13
1
1
1
1
5
D
E
6
5
F
G
7
1
H
2


A
5
B
11
C
8
D
4
E
F
G
H
I
7
1
3
3
1
J

- **S** (Start) can lead to **A**, **B**, or **C**.
  - **A** is an OR node with two alternative paths: **A1** or **A2**.
  - **B** is an AND node that requires the completion of both **B1** and **B2**.
  - **C** directly leads to the goal **G**.
- **A1** and **A2** both lead to **A3**, which is an AND node that must complete two sub-tasks **A3.1** and **A3.2** before reaching **G**.
- **B1** leads to **B3** directly, and **B2** leads to **B4**, which is an OR node with two alternatives: **B4.1** or **B4.2**.

Costs:

- **S** → **A**: 12 units
- **S** → **B**: 10 units
- **S** → **C**: 20 units
- **A1** → **A3**: 15 units
- **A2** → **A3**: 18 units
- **A3.1** → **G**: 10 units
- **A3.2** → **G**: 5 units
- **B1** → **B3**: 8 units
- **B2** → **B4**: 7 units
- **B4.1** → **G**: 12 units
- **B4.2** → **G**: 14 units
- **C** → **G**: 15 units

The costs for expanding nodes are as follows:

- Node `B` leads to nodes `E` and `F` with costs 3 and 2, respectively.
- Node `C` leads to node `G` with a cost of 4.
- Node `D` leads to nodes `H` and `F` with costs 3 and 5, respectively.

The goal is to find the minimum cost path from the start node `A` to the goal node, where the goal nodes are `E`, `F`, `G`, and `H`.

AO* algorithm – Artificial intelligence

- In the above figure, the buying of a car may be broken down into smaller problems or tasks that can be accomplished to achieve the main goal in the above figure, which is an example of a simple AND-OR graph. The other task is to either steal a car that will help us accomplish the main goal or use your own money to purchase a car that will accomplish the main goal.
- The AND symbol is used to indicate the AND part of the graphs, which refers to the need that all subproblems containing the AND to be resolved before the preceding node or issue may be finished.
- The start state and the target state are already known in the knowledge-based search strategy known as the AO* algorithm, and the best path is identified by heuristics. The informed search technique considerably reduces the algorithm's time complexity. The AO* algorithm is far more effective in searching AND-OR trees than the A* algorithm.

AO* vs A*

1. Purpose and Problem Structure

- *AO Algorithm:**
  - **Used for**: Solving problems represented as AND-OR graphs.
  - **Problem Structure**: Suitable for problems where the solution can be a combination of different strategies or sub-solutions (AND-OR graphs).
- *A Algorithm:**
  - **Used for**: Solving single-agent pathfinding problems.
  - **Problem Structure**: Suitable for problems that can be represented as a single sequence of decisions or moves (a simple graph).

AO* vs A*

. Optimality

- *AO Algorithm:**
  - **Optimality**: Finds the optimal solution considering both AND and OR conditions, but its optimality depends on the accuracy of the heuristic.
  - **Efficiency**: The algorithm may need to backtrack if a previously chosen path turns out to be non-optimal after further exploration.
- *A Algorithm:**
  - **Optimality**: Guarantees finding the optimal path if the heuristic is admissible (never overestimates the cost).
  - **Efficiency**: A* is considered optimal and complete if the heuristic is both admissible and consistent.

AO* vs A*

. Complexity

- *AO Algorithm:**
  - **Complexity**: More complex due to the need to handle both AND and OR nodes, and the need to maintain multiple paths that might need to be explored simultaneously.
  - **Space and Time**: The complexity depends on the structure of the AND-OR graph and the quality of the heuristic.
- *A Algorithm:**
  - **Complexity**: Generally less complex than AO* as it deals with a simpler graph structure.
  - **Space and Time**: Exponential in the worst case, as A* may explore many nodes, but it is highly efficient with a good heuristic.

THANK - YOU